Standardize Power Platform Secrets in Azure Key Vault

Implementation Solution Guide

Version 1.0.0 · 2026-06-30

Contents

Executive Summary — Standardize Power Platform Secrets in Azure Key Vault 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Standardize Power Platform Secrets in Azure Key Vault 5

The problem this solves . . . 5

Secret types in scope . . . 5

The retrieval patterns (and how to choose) . . . 5

Access model . . . 6

Governance & rotation model . . . 7

What "no exposure" means (validated later) . . . 7

The deployment model . . . 7

2. Requirements Guide — Standardize Power Platform Secrets in Azure Key Vault 7

2.1 An Azure subscription with an Azure Key Vault . . . 7

2.2 Key Vault uses the Azure RBAC permission model . . . 8

2.3 The Power Platform / Dataverse service principal has Key Vault read access . . . 8

2.4 A Power Platform environment with Dataverse . . . 8

2.5 Maker access and a solution to hold the components . . . 8

2.6 Licensing for the retrieval pattern chosen . . . 8

2.7 An Entra identity for the connector / Function pattern (if used) . . . 9

2.8 Network access to Key Vault . . . 9

2.9 An inventory of current hardcoded secrets . . . 9

Checklist . . . 9

3. Implementation Guide — Standardize Power Platform Secrets in Azure Key Vault 10

Step 1 — Create and configure the Key Vault . . . 10

Step 2 — Store the secrets in the vault . . . 10

Step 3 — Grant the Power Platform service principal read access . . . 11

Step 4 — Create the solution and a Secret environment variable (Pattern A) . . . 11

Step 5 — Consume the secret (remove the hardcoded value) . . . 11

Step 6 — (Optional, Pattern B) Wire the Azure Key Vault connector . . . 12

Step 7 — (Optional, Pattern C) Azure Function + managed identity via custom connector . . . 12

Step 8 — Set expiry, rotation, and ownership (governance) . . . 12

Step 9 — Validate no exposure (pilot gate) . . . 12

Step 10 — Expand across the environment . . . 13

5. Testing Guide — Standardize Power Platform Secrets in Azure Key Vault 13
Test 1 — Vault permission model is RBAC . . . 13
Test 2 — Power Platform service principal has read access . . . 13
Test 3 — The Secret environment variable resolves . . . 13
Test 4 — Secret value NOT in flow run history . . . 13
Test 5 — Secret value NOT in app formulas / flow definition . . . 14
Test 6 — Secret value NOT in connector definitions or exports . . . 14
Test 7 — Rotation flows through without a rebuild . . . 14
Test 8 — Inventory / access / expiry report . . . 14
Pass/fail gate . . . 14
6. Troubleshooting Guide — Standardize Power Platform Secrets in Azure Key Vault 14
A. Secret value showing in flow run history . . . 14
B. Env-var / secret not resolving (RBAC or permission-model mismatch) . . . 15
C. Env var configured but returns empty / old value . . . 15
D. Connector / Function authentication failures (Patterns B and C) . . . 16
E. Vault firewall / networking . . . 16
F. Hardcoded secret still found in an export . . . 16
G. Diagnostics quick reference . . . 16
H. When to escalate . . . 17
7. Best Practices — Standardize Power Platform Secrets in Azure Key Vault 17
8. Deliverables Summary — Standardize Power Platform Secrets in Azure Key Vault 18
Documentation . . . 18
PowerShell scripts . . . 18
What the buyer receives . . . 18
Outcome . . . 19
Appendix A — PowerShell Scripts 20

Executive Summary — Standardize Power Platform Secrets in Azure Key Vault

Power Platform makes it far too easy to paste an API key into a flow, a client secret into a custom connector, or a connection string into an app formula — and once a secret is embedded that way it leaks into **flow run history**, app and flow definitions, connector exports, and logs, and it cannot be rotated without rebuilding every app and flow that hardcoded it. This solution moves every Power Platform secret into **Azure Key Vault**, retrieves it securely through an approved pattern, removes the hardcoded copies, proves no exposure remains, and governs rotation, ownership, and access — so a key change never means an app rebuild.

This is a complete, **deploy-it-today** package for standardizing Power Platform secrets in Azure Key Vault, written for an administrator with no prior secrets-management experience. It turns scattered, hardcoded credentials into a single vaulted store consumed through the Microsoft-recommended pattern and secured by Entra ID RBAC.

What you get

- A plain-language **Solution Overview** — the secret types in scope, the four retrieval patterns

(Key Vault-backed **Secret environment variables**, the **Azure Key Vault connector**, an **Azure Function + managed identity** via a custom connector, and direct service-principal/managed-identity retrieval) with a clear **decision guide**, plus the access and governance/rotation model.

- A **Requirements Guide** with why / verify / implement-if-missing for every prerequisite

(subscription and vault, the RBAC permission model, the Power Platform service principal, Dataverse, maker access, licensing, Entra identities, networking, and the hardcoded-secret inventory).

- An **Implementation Guide** of precise, ordered steps across the Azure portal, the Power Platform

admin center, and the Power Apps maker portal — create and lock down the vault, store the secrets, grant the Power Platform service principal `Key Vault Secrets User`, wire the Secret environment variable, refactor apps/flows/connectors off hardcoded values, govern rotation, and validate no exposure — then pilot, validate, and expand.

- Five **PowerShell scripts** for Key Vault + secret + RBAC discovery (with a hardcoded-secret scan

of exports), access/retrieval validation, a secrets inventory + expiry/rotation report, safe remediation, and a rotation/ownership-tagging helper.

- **Testing** and **Troubleshooting** guides for a confident rollout.

What this delivers

A **vaulted, governed secrets model** for Power Platform: secrets live in Azure Key Vault, apps and flows read them through a Key Vault-backed Secret environment variable (or the connector / Function pattern where it fits), access is least-privilege via Entra ID RBAC, no secret value survives in run history, formulas, connectors, or exports, and rotating a secret in the vault flows through to every consumer **without an app rebuild** — proven on a pilot before you scale.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with a pilot gate before production.

Outcome

Every Power Platform secret centralized in Azure Key Vault, retrieved securely through approved patterns, with no hardcoded credentials left anywhere — and rotation that never requires an app rebuild — deployable by a single administrator and validated before rollout.

1. Solution Overview — Standardize Power Platform Secrets in Azure Key Vault

The problem this solves

Power Platform makes it trivial to paste a secret where it does not belong: an API key in a Power Automate flow action, a client secret in a custom connector, a connection string in a Power Apps formula, a service-account password in an environment variable, or a token in a script. Once a secret is embedded that way it leaks — into **flow run history** (inputs/outputs), **app and flow definitions**, **connector exports**, **solution exports**, and **logs** — and it cannot be rotated without editing and re-publishing every app and flow that hardcoded it. This solution moves every Power Platform secret into **Azure Key Vault**, retrieves it through an approved pattern, removes the hardcoded copies, proves no exposure remains, and governs rotation, ownership, and access — so a key change never means an app rebuild.

Secret types in scope

API keys, OAuth client secrets, connection strings, certificates (and certificate passwords), service-account credentials, external-system tokens, and any integration credential that today lives inside a Power App, a Power Automate flow, a custom connector, a Dataverse environment variable of a plaintext type, or a script.

The retrieval patterns (and how to choose)

There is more than one supported way to read a Key Vault secret from Power Platform. Lead with the native pattern; use the alternatives only where they fit.

Pattern A (PRIMARY) — Secret environment variables backed by Azure Key Vault

A Dataverse **environment variable of type** `Secret` stores only a **reference** (the Key Vault secret's resource identifier), not the value. At run time Power Apps / Power Automate resolves the reference and reads the current secret value through the Power Platform / Dataverse first-party service principal, which you grant the Key Vault `Key Vault Secrets User` RBAC role. This is the **Microsoft-recommended** approach: the value never lands in the app/flow definition, it moves cleanly between environments as part of a solution, and rotating the vault secret is picked up automatically because the reference resolves the **current** version.

- **Use when:** you want the standard, lowest-friction, no-code pattern — which is most cases.

- **Requires:** the vault on the **Azure RBAC** permission model, the Power Platform service principal granted `Key Vault Secrets User` on the vault (or secret), and the secret's resource ID.

Pattern B — The Azure Key Vault connector (Power Automate)

The built-in **Azure Key Vault** connector has a `Get secret` action. It is a **premium** connector (licensing applies) and authenticates as a **service principal / app registration** you configure on the connection.

- **Use when:** a flow needs a secret ad hoc and you accept a premium connector; or you need actions the env-var pattern does not cover.
- **Caution:** the returned value is a flow variable — if the action or a downstream action is not marked **secure**, the value appears in run history. Always enable **Secure Outputs** on the `Get secret` action and **Secure Inputs** on any action that consumes it.

Pattern C — Azure Function + managed identity, called via a custom connector

An **Azure Function** with a **system-assigned managed identity** reads the secret from Key Vault (the managed identity is granted `Key Vault Secrets User`) and returns only the result of whatever work needs the secret (or the value over a secured channel). A **custom connector** exposes the Function to flows/apps.

- **Use when:** you need transformation, want the secret to never leave Azure, need tighter control or auditing, or must broker calls to an external API without handing the secret to Power Platform.
- **Trade-off:** more moving parts (Function + connector) to build and maintain.

Pattern D — Direct service principal / managed identity retrieval

A script or component authenticates as a **service principal** or **managed identity** and reads the secret directly (e.g. Azure PowerShell, Azure CLI, or an SDK).

- **Use when:** the consumer is a script or an Azure-hosted component outside the Power Platform no-code surface. Not a Power Apps/Automate retrieval pattern per se — included for completeness.

> ***Decision guide (short form):*** *default to **Pattern A** (Secret environment variables). Reach for **Pattern B** only when a premium connector is acceptable and the env-var pattern will not do. Choose **Pattern C** when you need transformation, brokering, or the secret to stay inside Azure. Use **Pattern D** for scripts / Azure-hosted consumers.*

Access model

Access to the vault is granted with **Entra ID identities** — app registrations / service principals and **managed identities** — assigned **Azure RBAC** roles on the Key Vault, least privilege:

- `Key Vault Secrets User` — read secret values (what a consumer needs).
- `Key Vault Secrets Officer` — create/update/rotate secrets (what a secret owner/administrator needs).

Prefer the **Azure RBAC** permission model over the legacy **access-policy** model — RBAC is role-based, auditable, and consistent with the rest of Azure. Grant the read role at the **secret** scope where you can, and at the vault scope only when a consumer legitimately needs the whole vault.

Governance & rotation model

Secrets are only as safe as their lifecycle. This solution establishes:

- **Ownership** — every secret is tagged with an owner and a purpose so accountability is explicit.
- **Least-privilege access** — only the identities that must read a secret hold `Key Vault Secrets User; only owners hold Key Vault Secrets Officer`; everything else is denied by default.
- **Expiry & rotation** — every secret has an expiration and a rotation cadence; expired or soon-to-expire secrets are surfaced in the report so nothing silently ages out.
- **Rotation without a rebuild** — this is the payoff of vaulting. When you rotate a secret (create a new version in the vault), the **Secret environment variable** (Pattern A) and the **connector / Function** (Patterns B/C) resolve the **current** version at run time. Apps and flows keep working with the new value **without any edit or re-publish**. (A reference pinned to a specific version is the exception — reference the secret without a version to always get current.)

What "no exposure" means (validated later)

A secret is *out* of Power Platform only when its value appears in **none** of: flow run history inputs/outputs, app formulas, connector definitions/exports, solution exports, or logs. The Testing Guide proves each of these; the Discovery script scans exported definitions for hardcoded-secret patterns as a safety net.

The deployment model

> ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand***, *with a* ***pilot gate****: prove the pattern on a single flow or app, validate no exposure, then expand across the environment. A rollback note is included at each disruptive step.*

2. Requirements Guide — Standardize Power Platform Secrets in Azure Key Vault

Verify **every** requirement before configuring. Each entry: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites are in `global-requirements-azure-solutions.md`.

> *Object naming (reused verbatim across docs and scripts): - Key Vault:* `PP-Secrets – <Environment> – Vault` *- Secret environment variable:* `Secret EnvVar – <Integration> – <Environment>` *- Read role assignment:* `Secrets Reader – Power Platform SP – <Vault>` *- App registration / managed identity (connector/Function):* `Secrets Reader – <Component> – <Environment>` *- Custom connector:* `KV Broker – <Integration> – <Environment>` *- Azure Function:* `KV Broker Function – <Integration> – <Environment>`

2.1 An Azure subscription with an Azure Key Vault

Description: An Azure subscription in scope and a Key Vault to hold the secrets (`PP-Secrets – <Environment> – Vault`). **Why required:** The vault is the single store for all Power Platform secrets.

How to verify: Azure portal → **Key vaults**; confirm the vault exists (or the subscription can create one). Run `PPKV-Discovery.ps1` to list vaults. **How to implement if missing:** Create a Key Vault (Implementation Step 1). One vault per Power Platform environment is the clean default.

2.2 Key Vault uses the Azure RBAC permission model

Description: The vault's permission model is set to **Azure role-based access control**, not the legacy **Vault access policy** model. **Why required:** The Secret environment variable pattern and least-privilege role grants assume RBAC; a vault on the access-policy model will not honor the RBAC role assignments this solution uses. **How to verify:** Vault → **Settings** → **Access configuration** shows **Azure role-based access control**. Run `PPKV-Validation.ps1` (reports the permission model). **How to implement if missing:** Switch **Access configuration** → **Permission model** to **Azure role-based access control** (Implementation Step 1).

2.3 The Power Platform / Dataverse service principal has Key Vault read access

Description: The first-party Power Platform / Dataverse service principal is granted the `Key Vault Secrets User` RBAC role on the vault (or on the specific secrets) — the role assignment `Secrets Reader – Power Platform SP – <Vault>`. **Why required:** Pattern A (Secret environment variables) resolves the vault reference **as** this service principal; without the read role the environment variable will not resolve. **How to verify:** Vault → **Access control (IAM)** → **Role assignments**; confirm `Key Vault Secrets User` is assigned to the Power Platform service principal. Run `PPKV-Validation.ps1`. **How to implement if missing:** Assign the role (Implementation Step 3) or run `PPKV-Remediation.ps1` to assign `Key Vault Secrets User` to the service principal.

2.4 A Power Platform environment with Dataverse

Description: The target Power Platform environment has **Dataverse** provisioned (required for solutions and environment variables of type Secret). **Why required:** Secret environment variables are Dataverse components; without Dataverse there are no solutions and no Secret env-var type. **How to verify:** Power Platform admin center (`https://admin.powerplatform.microsoft.com`) → **Environments** → the environment shows a **Dataverse** database. **How to implement if missing:** Add a Dataverse database to the environment in the Power Platform admin center.

2.5 Maker access and a solution to hold the components

Description: A maker account with rights to the target environment and a **solution** in the Power Apps maker portal to contain the Secret environment variable(s). **Why required:** Environment variables (Secret type) are authored inside a solution in the maker portal. **How to verify:** Power Apps maker portal (`https://make.powerapps.com`) → select the environment → **Solutions**; confirm a solution exists (or can be created) and you can edit it. **How to implement if missing:** Create a solution (Implementation Step 4); ensure the maker holds at least environment-maker rights.

2.6 Licensing for the retrieval pattern chosen

Description: Licensing sufficient for the pattern in use. Pattern A (Secret env vars) uses standard Power Platform capability. Pattern B (Azure Key Vault connector) is a **premium** connector requiring premium Power Platform licensing. Pattern C adds an **Azure Function** (Azure consumption cost). **Why required:** Premium connectors and Azure resources are gated by licensing/subscription cost. **How to verify:** Power Platform admin center → **Billing / Licenses**; confirm premium entitlement if using Pattern B. Confirm the Azure subscription can host a Function for Pattern C. **How to implement if missing:** Assign premium Power Platform licensing for Pattern B, or default to Pattern A (no premium requirement).

2.7 An Entra identity for the connector / Function pattern (if used)

Description: For Pattern B, an **app registration / service principal** (`Secrets Reader – <Component> – <Environment>`) for the connector's connection. For Pattern C, a **managed identity** on the Azure Function. **Why required:** The connector authenticates as a service principal; the Function reads the vault as its managed identity. Each needs `Key Vault Secrets User`. **How to verify:** Entra → **App registrations** (Pattern B) or the Function's **Identity** blade (Pattern C); confirm the identity exists and holds `Key Vault Secrets User` on the vault (`PPKV-Validation.ps1`). **How to implement if missing:** Register the app / enable the managed identity, then assign `Key Vault Secrets User` (Implementation Step 6/7). Skip if using Pattern A only.

2.8 Network access to Key Vault

Description: Power Platform / Azure Function egress can reach the Key Vault data-plane endpoint (`https://<vault>.vault.azure.net`); vault firewall / private endpoint allows the consumer. **Why required:** If the vault firewall blocks the caller, secret resolution fails even with correct RBAC. **How to verify:** Vault → **Networking**; confirm the caller is permitted (public access, selected networks including trusted Microsoft services, or private endpoint reachable). **How to implement if missing:** Adjust the vault firewall to allow the required networks / trusted services, or add a private endpoint (Implementation Step 1).

2.9 An inventory of current hardcoded secrets

Description: A list of where secrets are hardcoded today — flow actions, app formulas, custom connector definitions, plaintext environment variables, scripts. **Why required:** You cannot remove what you have not found; the inventory scopes the remediation. **How to verify:** Run `PPKV-Discovery.ps1` against **exported** flow/app/connector JSON to flag hardcoded-secret patterns; review manually. **How to implement if missing:** Export the solution(s) / flows / apps and run the discovery scan; record each finding for remediation.

Checklist

- Azure subscription + Key Vault `PP-Secrets – <Environment> – Vault`
- Vault on the **Azure RBAC** permission model — `PPKV-Validation.ps1`
- Power Platform service principal has `Key Vault Secrets User` — `PPKV-Validation.ps1`
- Power Platform environment with Dataverse
- Maker access + a solution to hold the Secret env var

- Licensing for the chosen pattern (premium if using the connector)
- Entra identity + read role for connector/Function pattern (if used)
- Network path to the vault open for the consumer
- Hardcoded-secret inventory captured — `PPKV-Discovery.ps1`

3. Implementation Guide — Standardize Power Platform Secrets in Azure Key Vault

Follow these steps **in order**. Work spans three portals:

- **Azure portal** (`https://portal.azure.com`) — Key Vault and RBAC.
- **Power Platform admin center** (`https://admin.powerplatform.microsoft.com`) — environment.
- **Power Apps maker portal** (`https://make.powerapps.com`) — solution and Secret environment variable.

> ***Model:*** *Prove the pattern on* ***one*** *flow or app first (the pilot gate), validate no exposure, then expand.* ***Pattern A (Secret environment variables) is the default path****; Steps 6–7 cover the optional connector / Function patterns. Object names below match* `02-requirements.md`.

Step 1 — Create and configure the Key Vault

1. Azure portal → search **Key vaults** → **Create**.
2. **Subscription / Resource group:** select in-scope. **Key vault name:** `PP-Secrets – <Environment> – Vault` (use a DNS-safe form, e.g. `pp-secrets-prod-kv`). **Region:** your standard.
3. On the **Access configuration** tab, set **Permission model** to **Azure role-based access control**. (This satisfies requirement 2.2.)
4. On the **Networking** tab, choose public access, selected networks (allow trusted Microsoft services), or a private endpoint per requirement 2.8.
5. **Review + create** → **Create.**

> *Rollback note: a newly created vault holds no secrets and grants no access until you add them in the next steps — deleting it (soft-delete honored) fully reverses Step 1.*

Step 2 — Store the secrets in the vault

1. Open `PP-Secrets – <Environment> – Vault` → **Objects** → **Secrets** → **+ Generate/Import**.
2. **Upload options:** Manual. **Name:** a stable, purpose-based name (e.g. `crm-api-key`, `sftp-connection-string`). **Value:** paste the secret value. Set an **Expiration date** (governance — see Step 8).
3. **Create.** Repeat for each secret from the inventory (requirement 2.9).

4. Copy each secret's **Secret Identifier** (the resource URI). Use the **versionless** form (without the trailing version GUID) so consumers always resolve the current version.

Do not record secret values in documentation, tickets, or scripts — only the names and identifiers.

Step 3 — Grant the Power Platform service principal read access

1. Vault → **Access control (IAM)** → **+ Add** → **Add role assignment**.

2. **Role:** `Key Vault Secrets User`. **Next.**

3. **Assign access to:** User, group, or service principal → **+ Select members** → search the first-party **Power Platform** / **Dataverse** service principal (the Microsoft first-party app used by Power Platform to resolve Key Vault references). Select it.

4. **Review + assign.** Name this assignment (for your records) `Secrets Reader – Power Platform SP – <Vault>`.

5. Scope tip: to grant per-secret instead of vault-wide, add the role assignment from the individual **secret's** Access control (IAM) blade.

If you are unsure which service principal object represents Power Platform in your tenant, run `PPKV-Validation.ps1` — it identifies the Power Platform / Dataverse service principal and reports whether it already holds `Key Vault Secrets User`.

Step 4 — Create the solution and a Secret environment variable (Pattern A)

1. Power Apps maker portal → select the target environment (top right) → **Solutions**.

2. **+ New solution** → **Display name:** a solution to hold the components → set/choose a **Publisher** → **Create**. (Or open an existing solution.)

3. Inside the solution: **+ New** → **More** → **Environment variable** (the exact maker menu path is volatile — if the label differs, choose **Environment variable**; defer UI minutiae to Microsoft docs).

4. **Display name:** `Secret EnvVar – <Integration> – <Environment>`. **Data type: Secret**. **Secret store: Azure Key Vault**.

5. Provide the Key Vault secret reference: the **Azure subscription**, the **Key Vault name** (`PP-Secrets – <Environment> – Vault`), and the **secret name** from Step 2 (the versionless reference). **Save.**

Rollback note: deleting the environment variable removes only the reference — the vault secret is untouched.

Step 5 — Consume the secret (remove the hardcoded value)

1. **In a Power Automate flow:** open the flow, add the environment variable's current value where the secret was hardcoded. Reference the environment variable (via the environment-variable value, e.g. the Dataverse/dynamic-content reference for `Secret EnvVar – <Integration> – <Environment>`) instead of the literal secret. Delete the literal.

2. **In a Power App:** replace the literal in the formula with a reference to the environment variable (retrieved through a flow or the supported environment-variable access, per Microsoft docs — apps should obtain the secret via a flow rather than reading the vault directly).

3. **Mark actions secure:** on any Power Automate action that receives or passes the secret, open **Settings** and enable **Secure Inputs** and **Secure Outputs** so the value never appears in run history.

4. **Save and publish** the flow/app.

Rollback note: the pre-change flow/app definition captured under requirement G6 is the back-out; re-import it if the vaulted path misbehaves during the pilot.

Step 6 — (Optional, Pattern B) Wire the Azure Key Vault connector

1. Register an Entra app `Secrets Reader – <Component> – <Environment>` (Entra → **App registrations** → **New registration**); create a client secret or certificate for it.

2. Assign it `Key Vault Secrets User` on the vault (as in Step 3).

3. In a flow, add the **Azure Key Vault** connector's **Get secret** action; create the connection using the service principal (tenant ID, client ID, credential).

4. On the **Get secret** action → **Settings** → **enable Secure Outputs**; enable **Secure Inputs** on any consuming action.

Use only when the env-var pattern will not do; this is a premium connector (requirement 2.6).

Step 7 — (Optional, Pattern C) Azure Function + managed identity via custom connector

1. Create an **Azure Function** `KV Broker Function – <Integration> – <Environment>`; on its **Identity** blade enable the **system-assigned managed identity**.

2. Assign the managed identity `Key Vault Secrets User` on the vault (as in Step 3).

3. In the Function code, read the secret from Key Vault using the managed identity (e.g. `DefaultAzureCredential` + the Key Vault secret client) and return only the required result.

4. Build a **custom connector** `KV Broker – <Integration> – <Environment>` that calls the Function; secure the connector's authentication. Call it from the flow/app.

Use when you need transformation, brokering, or the secret to stay entirely inside Azure.

Step 8 — Set expiry, rotation, and ownership (governance)

1. For each secret, confirm an **Expiration date** (Step 2) and apply **tags** for owner and purpose. Run `PPKV-Set-RotationOwnership.ps1` to set expiry and ownership/rotation tags consistently across secrets.

2. Establish the rotation cadence per the Best Practices guide. Because consumers reference the **versionless** secret, rotating (adding a new secret version) is picked up automatically — no app/flow edit required.

Step 9 — Validate no exposure (pilot gate)

1. Run `PPKV-Validation.ps1` → confirms the vault is on the RBAC model, the Power Platform service principal has `Key Vault Secrets User`, and a Secret environment variable resolves.

2. Run the flow/app once, then open the flow's **run history** and confirm the secret value is **not** visible in inputs/outputs (Secure Inputs/Outputs working). Complete the **Testing Guide**.

3. Run `PPKV-Discovery.ps1` against fresh **exports** of the pilot flow/app/connector to confirm no hardcoded-secret pattern remains.

Step 10 — Expand across the environment

1. Repeat Steps 2, 4, 5 (and 6/7 where used) for the remaining secrets/flows/apps from the inventory.

2. Re-run `PPKV-Report.ps1` for a full secrets inventory with expiry/rotation and access, and `PPKV-Discovery.ps1` for a final hardcoded-secret sweep of all exports.

Order of operations

`Create + RBAC vault → Store secrets → Grant PP SP read → Solution + Secret env var → Refactor consumers (mark secure) → (connector/Function if needed) → Governance (expiry/rotation/ownership) → Validate no exposure → Expand`

5. Testing Guide — Standardize Power Platform Secrets in Azure Key Vault

Run after the pilot configuration applies (Implementation Step 9) and again after each expansion and after each rotation. The goal of the last four tests is to prove the secret value appears **nowhere** in Power Platform.

Test 1 — Vault permission model is RBAC

Do: Azure portal → vault → **Access configuration**, or run `PPKV-Validation.ps1`. **Expected:** Permission model is **Azure role-based access control**.

Test 2 — Power Platform service principal has read access

Do: Vault → **Access control (IAM)** → **Role assignments**, or run `PPKV-Validation.ps1`. **Expected:** `Key Vault Secrets User` is assigned to the Power Platform / Dataverse service principal (`Secrets Reader – Power Platform SP – <Vault>`).

Test 3 — The Secret environment variable resolves

Do: Run the pilot flow/app that references `Secret EnvVar – <Integration> – <Environment>`; run `PPKV-Validation.ps1`. **Expected:** The flow/app runs successfully using the vault-backed value — the reference resolves to the current secret version.

Test 4 — Secret value NOT in flow run history

Do: Open the flow's **run history** → the run → inspect each action's inputs and outputs. **Expected:** The secret value is masked / not shown; actions that touch the secret show **Secure Inputs/Outputs** in effect. No literal secret anywhere in the run.

Test 5 — Secret value NOT in app formulas / flow definition

Do: Inspect the app formulas and the flow actions where the secret was previously hardcoded. **Expected:** Only a reference to the environment variable remains — no literal secret value.

Test 6 — Secret value NOT in connector definitions or exports

Do: Export the solution (and any custom connector). Open the exported JSON; run `PPKV-Discovery.ps1` against the export. **Expected:** No hardcoded-secret pattern is flagged; the exported definition contains only references, not values.

Test 7 — Rotation flows through without a rebuild

Do: Add a new version of the secret in the vault (or run `PPKV-Set-RotationOwnership.ps1` to update expiry), then run the flow/app again **without editing it**. **Expected:** The flow/app uses the **new** value automatically (versionless reference resolves current) — no edit or re-publish was needed.

Test 8 — Inventory / access / expiry report

Do: Run `PPKV-Report.ps1`. **Expected:** A clear list of secrets with owner/purpose tags, expiry/rotation status, and who holds read/manage roles — no secret values printed.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–6 pass on the pilot flow/app — the vault-backed value works and appears in no run history, formula, connector, or export
Rotation pass	Test 7 passes — a rotated secret is picked up with no rebuild
Expand pass	Tests 3–6 pass for every additional flow/app, and Test 8 shows a clean, governed inventory

6. Troubleshooting Guide — Standardize Power Platform Secrets in Azure Key Vault

Use `PPKV-Validation.ps1` for access/permission-model state, `PPKV-Discovery.ps1` for the vault/secret inventory and a hardcoded-secret scan of exports, `PPKV-Report.ps1` for expiry/rotation and access, and `PPKV-Remediation.ps1` for safe fixes.

A. Secret value showing in flow run history

Symptom	Cause	Remediation
The literal secret appears in an action's inputs/outputs in run history	Action not marked secure	Open the action → **Settings** → enable **Secure Inputs** and **Secure Outputs**; enable Secure Inputs on every downstream action that receives the value; re-run and re-check
Value still visible after enabling secure on one action	A different action (or a compose/parse) re-exposes it	Trace the value through every action; mark each secure. A value passed into an unsecured action re-appears
Value in a variable/compose output	The secret was copied into an unsecured Compose/Set variable	Remove the intermediate, or mark that action secure; prefer passing the env-var reference, not the value

B. Env-var / secret not resolving (RBAC or permission-model mismatch)

Symptom	Cause	Remediation
Flow/app fails resolving the Secret environment variable	Power Platform service principal lacks `Key Vault Secrets User`	Assign the role (Implementation Step 3) or run `PPKV-Remediation.ps1`; confirm with `PPKV-Validation.ps1`
Role assigned but still denied	Vault is on the **legacy access-policy** model, so RBAC roles are ignored	Switch **Access configuration** → **Permission model** to **Azure RBAC** (requirement 2.2); re-test
Resolves in one environment, not another	Wrong vault/secret reference or a different service principal per environment	Confirm the env var points at the correct vault/secret for that environment; verify the SP has the role on **that** vault
Intermittent failures	RBAC assignment still propagating	Allow a few minutes for the role assignment to propagate, then retry

C. Env var configured but returns empty / old value

Symptom	Cause	Remediation
Returns an old value after rotation	The reference is pinned to a specific secret **version**	Re-point the env var / connector at the **versionless** secret reference so it resolves current
Returns empty	Secret expired or disabled in the vault	Check the secret's status/expiry (`PPKV-Report.ps1`); enable / add a new version; set a future expiry with `PPKV-Set-RotationOwnership.ps1`

Symptom	Cause	Remediation
Reference invalid	Secret renamed or moved	Update the env-var reference to the current secret name/identifier

D. Connector / Function authentication failures (Patterns B and C)

Symptom	Cause	Remediation
Azure Key Vault connector `Get secret` returns 403	The connection's service principal lacks `Key Vault Secrets User`	Assign the role to the app registration `Secrets Reader – <Component> – <Environment>`; recreate the connection if the credential expired
Connector connection fails to create	Wrong tenant/client ID or an expired client secret	Verify the app registration's IDs; generate a fresh client secret/certificate and update the connection
Function returns 403 from Key Vault	Managed identity not enabled or not granted read	Enable the Function's system-assigned identity; assign it `Key Vault Secrets User` on the vault
Function/connector call blocked	Vault firewall denies the caller	Allow the caller's network / trusted Microsoft services, or use a private endpoint (requirement 2.8)

E. Vault firewall / networking

- If RBAC is correct but resolution still fails, check the vault **Networking** blade — a restrictive

firewall blocks the caller regardless of role. Allow the required networks / trusted Microsoft services, or reach the vault over a private endpoint.

F. Hardcoded secret still found in an export

- `PPKV-Discovery.ps1` flags a pattern in exported JSON → open that flow/app/connector, replace the

literal with the environment-variable reference (Implementation Step 5), re-export, and re-scan until clean. Treat any finding as unremediated.

G. Diagnostics quick reference

```
# Vault permission model + read-role check for the Power Platform SP
.\PPKV-Validation.ps1 -VaultName 'pp-secrets-prod-kv'
# Inventory vaults/secrets/RBAC + scan an export folder for hardcoded-secret patterns
.\PPKV-Discovery.ps1 -ExportPath 'C:\exports\pilot'
# Expiry / rotation / access report (no values printed)
.\PPKV-Report.ps1 -VaultName 'pp-secrets-prod-kv'
```

H. When to escalate

If the vault is on the RBAC model, the Power Platform service principal holds `Key Vault Secrets User`, the network path is open, the secret is enabled and referenced versionless, and the env var still will not resolve, capture the `PPKV-Validation.ps1` output and the exact error and open a Microsoft support case (Power Platform + Key Vault).

7. Best Practices — Standardize Power Platform Secrets in Azure Key Vault

- **Default to Secret environment variables.** Pattern A keeps the value out of the app/flow definition, travels with the solution between environments, and rotates without a rebuild — it is the Microsoft-recommended approach and the right choice in most cases.
- **Use the Azure RBAC permission model, not access policies.** RBAC is role-based, auditable, and consistent with the rest of Azure; the Secret env-var pattern assumes it.
- **Least privilege, per secret where you can.** Grant `Key Vault Secrets User` (read) only to the identities that must read, and `Key Vault Secrets Officer` (manage) only to owners. Scope the read role to the individual secret rather than the whole vault when a consumer needs just one.
- **Reference secrets versionless.** Point env vars / connectors at the secret **without** a version so rotation is picked up automatically. Pin to a version only when you deliberately want to freeze it.
- **Always mark actions Secure.** Enable **Secure Inputs** and **Secure Outputs** on every Power Automate action that receives or passes a secret, so nothing lands in run history.
- **Never let apps read the vault directly.** Have Power Apps obtain a secret through a flow (or the Function/connector broker), not by embedding vault calls in formulas.
- **One vault per environment.** A vault per Power Platform environment keeps dev/test/prod secrets and access cleanly separated and makes solution promotion predictable.
- **Tag every secret with an owner and purpose.** Accountability and audit start with knowing who owns each secret and why it exists; enforce this with `PPKV-Set-RotationOwnership.ps1`.
- **Set expiry and a rotation cadence.** Every secret gets an expiration and a rotation schedule; chase expiring secrets from `PPKV-Report.ps1` before they lapse.
- **Prove no exposure, don't assume it.** After every change, verify the value is absent from run history, formulas, connector definitions, and exports (Testing Guide) — and scan exports with `PPKV-Discovery.ps1`.
- **Never hardcode again.** Make the vaulted pattern the standard for new flows/apps; keep the discovery scan in the review pipeline so a hardcoded secret is caught before it ships.

8. Deliverables Summary — Standardize Power Platform Secrets in Azure Key Vault

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The problem, secret types in scope, the four retrieval patterns + decision guide, access model, governance/rotation model
2. Requirements Guide	Every prerequisite with Why / Verify / Implement-if-missing
Global Requirements (Azure Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps across the Azure, Power Platform admin, and Power Apps maker portals
5. Testing Guide	Validation procedures with pass/fail gates (including no-exposure proofs)
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`PPKV-Discovery.ps1`	Inventories Key Vaults, secrets, and RBAC assignments; best-effort scan of exported flow/app/connector JSON for hardcoded-secret patterns (read-only)
`PPKV-Validation.ps1`	Confirms the Power Platform service principal has `Key Vault Secrets User`, a secret resolves, and the vault uses the RBAC model (read-only)
`PPKV-Report.ps1`	Secrets inventory with expiry/rotation status and read/manage access — no values printed (read-only)
`PPKV-Remediation.ps1`	Safe remediation (defaults to `-WhatIf`): assigns `Key Vault Secrets User`, sets secret expiry
`PPKV-Set-RotationOwnership.ps1`	Sets rotation/expiry and ownership tags on secrets (defaults to `-WhatIf`)

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Every Power Platform secret centralized in Azure Key Vault, retrieved through an approved pattern, with no hardcoded credentials left in apps, flows, connectors, exports, or logs — and rotation that never requires an app rebuild, proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PPKV-Discovery.ps1

Purpose: Read-only discovery for the "Standardize Power Platform Secrets in Azure Key Vault" solution. Lists Key Vaults in the subscription (or one named vault), their permission model, their secrets (metadata only), and the RBAC role assignments on each vault. If -ExportPath is supplied, performs a best-effort scan of exported flow / app / custom-connector JSON files for patterns that look like hardcoded secrets. Makes no changes and prints NO secret values. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PPKV-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\PPKV-Discovery.ps1 -VaultName 'pp-secrets-prod-kv' powershell -ExecutionPolicy Bypass -File .\PPKV-Discovery.ps1 -ExportPath 'C:\exports\pilot' Requires the Az PowerShell modules (Az.Accounts, Az.KeyVault, Az.Resources). Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A per-vault snapshot: permission model, secret names with enabled/expiry metadata, and RBAC role assignments. If -ExportPath is used, a list of files + line hints where a hardcoded secret pattern was detected (for manual review) — a single readiness picture before you vault and remediate. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. Operates on metadata/identifiers only — never reads or prints secret values. The export scan is heuristic (best-effort) and flags candidates for human review; it is not a guarantee. #> [CmdletBinding()] param([string]$SubscriptionId, [string]$VaultName, [string]$ExportPath)

How to run: powershell -ExecutionPolicy Bypass -File .\PPKV-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\PPKV-Discovery.ps1 -VaultName 'pp-secrets-prod-kv' powershell -ExecutionPolicy Bypass -File .\PPKV-Discovery.ps1 -ExportPath 'C:\exports\pilot' Requires the Az PowerShell modules (Az.Accounts, Az.KeyVault, Az.Resources). Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A per-vault snapshot: permission model, secret names with enabled/expiry metadata, and RBAC role assignments. If -ExportPath is used, a list of files + line hints where a hardcoded secret pattern was detected (for manual review) — a single readiness picture before you vault and remediate. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. Operates on metadata/identifiers only — never reads or prints secret values. The export scan is heuristic (best-effort) and flags candidates for human review; it is not a guarantee. #> [CmdletBinding()] param([string]$SubscriptionId, [string]$VaultName, [string]$ExportPath)

PPKV-Remediation.ps1

Purpose: Safe, targeted fixes for the "Standardize Power Platform Secrets in Azure Key Vault" solution. Two actions: 'GrantRead' — assign the 'Key Vault Secrets User' RBAC role to a principal (by object id or the Power Platform service principal) on the vault or a named secret; 'SetExpiry' — set an expiration date on a named secret (metadata only). DEFAULTS TO -WhatIf — it prints the change it would make and changes nothing until you pass -Execute. Never reads or prints secret values. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\PPKV-Remediation.ps1 -VaultName 'pp-secrets-prod-kv' -Action GrantRead -PrincipalObjectId '' powershell -ExecutionPolicy Bypass -File .\PPKV-Remediation.ps1 -VaultName 'pp-secrets-prod-kv' -Action SetExpiry -SecretName 'crm-api-key' -ExpiresOn '2027-01-01' Apply for real (explicit): ... -Execute Requires Az.Accounts, Az.KeyVault, Az.Resources. Run Connect-AzAccount first, or the script will prompt. Assigning roles requires Owner / User Access Administrator on the vault scope. EXPECTED RESULTS In -WhatIf mode (default): a printed

description of the role assignment or expiry change — nothing changes. With -Execute: the role is assigned, or the secret's expiry is set. .NOTES Workplace Marketplace — Azure Solutions. Defaults to -WhatIf. The repeatable way. SetExpiry updates metadata only — it does NOT read or change the secret value. GrantRead assigns read (not manage) access, per least privilege. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$VaultName, [Parameter(Mandatory)] [ValidateSet('GrantRead','SetExpiry')] [string]$Action, [string]$PrincipalObjectId, # for GrantRead (omit to target the Power Platform SP) [string]$SecretName, # for GrantRead (per-secret scope) or SetExpiry [datetime]$ExpiresOn, # for SetExpiry [string]$SubscriptionId, [switch]$Execute # omit for -WhatIf (default safe mode))

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\PPKV-Remediation.ps1 -VaultName 'pp-secrets-prod-kv' -Action GrantRead -PrincipalObjectId '<sp object id>' powershell -ExecutionPolicy Bypass -File .\PPKV-Remediation.ps1 -VaultName 'pp-secrets-prod-kv' -Action SetExpiry -SecretName 'crm-api-key' -ExpiresOn '2027-01-01' Apply for real (explicit): ... -Execute Requires Az.Accounts, Az.KeyVault, Az.Resources. Run Connect-AzAccount first, or the script will prompt. Assigning roles requires Owner / User Access Administrator on the vault scope. EXPECTED RESULTS In -WhatIf mode (default): a printed description of the role assignment or expiry change — nothing changes. With -Execute: the role is assigned, or the secret's expiry is set. .NOTES Workplace Marketplace — Azure Solutions. Defaults to -WhatIf. The repeatable way. SetExpiry updates metadata only — it does NOT read or change the secret value. GrantRead assigns read (not manage) access, per least privilege. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$VaultName, [Parameter(Mandatory)] [ValidateSet('GrantRead','SetExpiry')] [string]$Action, [string]$PrincipalObjectId, # for GrantRead (omit to target the Power Platform SP) [string]$SecretName, # for GrantRead (per-secret scope) or SetExpiry [datetime]$ExpiresOn, # for SetExpiry [string]$SubscriptionId, [switch]$Execute # omit for -WhatIf (default safe mode))

PPKV-Report.ps1

Purpose: Read-only reporting for the "Standardize Power Platform Secrets in Azure Key Vault" solution. For a named vault (or all vaults), lists every secret with its metadata — enabled state, created / updated dates, expiry, days-until-expiry, and owner/purpose tags — plus the RBAC role assignments (who can read vs. manage). Optionally writes a CSV. Prints NO secret values. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PPKV-Report.ps1 -VaultName 'pp-secrets-prod-kv' Optional: -CsvPath 'C:\reports\ppkv-secrets.csv' -ExpiringWithinDays 30 Requires Az.Accounts, Az.KeyVault, Az.Resources. Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A per-secret table (name, enabled, expiry, days left, owner/purpose tags) with expiring/ expired secrets highlighted, followed by the vault's read/manage role assignments — a clean governance snapshot. Optional CSV for records/audit. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. Operates on metadata/identifiers only — never reads or prints secret values. #> [CmdletBinding()] param([string]$VaultName, [string]$SubscriptionId, [string]$CsvPath, [int]$ExpiringWithinDays = 30)

How to run: powershell -ExecutionPolicy Bypass -File .\PPKV-Report.ps1 -VaultName 'pp-secrets-prod-kv' Optional: -CsvPath 'C:\reports\ppkv-secrets.csv' -ExpiringWithinDays 30 Requires Az.Accounts, Az.KeyVault, Az.Resources. Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A per-secret table (name, enabled, expiry, days left, owner/purpose tags) with expiring/ expired secrets highlighted, followed by the vault's read/manage role assignments — a clean governance snapshot. Optional CSV for records/audit. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. Operates on metadata/identifiers only — never reads or prints secret values. #> [CmdletBinding()] param([string]$VaultName, [string]$SubscriptionId, [string]$CsvPath,

[int]$ExpiringWithinDays = 30)

PPKV-Set-RotationOwnership.ps1

Purpose: Governance helper for the "Standardize Power Platform Secrets in Azure Key Vault" solution. For one named secret or every secret in a vault, sets an expiration date and applies consistent governance tags — 'owner', 'purpose', and 'rotationDays' (the intended rotation cadence) — so ownership and rotation are explicit and reportable (PPKV-Report.ps1 reads them). DEFAULTS TO -WhatIf — prints what it would change and changes nothing until you pass -Execute. Never reads or prints secret values; operates on metadata/tags only. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\PPKV-Set-RotationOwnership.ps1 -VaultName 'pp-secrets-prod-kv' -Owner 'integrations-team' -Purpose 'CRM API' -RotationDays 90 -SecretName 'crm-api-key' All secrets in a vault: ... -VaultName 'pp-secrets-prod-kv' -Owner 'integrations-team' -RotationDays 90 Apply for real (explicit): ... -Execute Requires Az.Accounts, Az.KeyVault. Run Connect-AzAccount first, or the script will prompt. Requires 'Key Vault Secrets Officer' (or equivalent) to update secret attributes. EXPECTED RESULTS In -WhatIf mode (default): per-secret lines showing the tags and expiry that would be set — nothing changes. With -Execute: each secret gets the owner/purpose/rotationDays tags and an expiry computed as (today + RotationDays) unless -ExpiresOn is given. .NOTES Workplace Marketplace — Azure Solutions. Defaults to -WhatIf. The repeatable way. Setting expiry/tags does NOT rotate the value — it records the cadence and ownership. Actual rotation is adding a new secret version; because consumers reference the versionless secret, a rotated value flows through with no app/flow rebuild. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$VaultName, [string]$SecretName, # omit to apply to all secrets in the vault [string]$Owner, [string]$Purpose, [int]$RotationDays = 90, [datetime]$ExpiresOn, # overrides the computed today+RotationDays expiry [string]$SubscriptionId, [switch]$Execute # omit for -WhatIf (default safe mode))

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\PPKV-Set-RotationOwnership.ps1 -VaultName 'pp-secrets-prod-kv' -Owner 'integrations-team' -Purpose 'CRM API' -RotationDays 90 -SecretName 'crm-api-key' All secrets in a vault: ... -VaultName 'pp-secrets-prod-kv' -Owner 'integrations-team' -RotationDays 90 Apply for real (explicit): ... -Execute Requires Az.Accounts, Az.KeyVault. Run Connect-AzAccount first, or the script will prompt. Requires 'Key Vault Secrets Officer' (or equivalent) to update secret attributes. EXPECTED RESULTS In -WhatIf mode (default): per-secret lines showing the tags and expiry that would be set — nothing changes. With -Execute: each secret gets the owner/purpose/rotationDays tags and an expiry computed as (today + RotationDays) unless -ExpiresOn is given. .NOTES Workplace Marketplace — Azure Solutions. Defaults to -WhatIf. The repeatable way. Setting expiry/tags does NOT rotate the value — it records the cadence and ownership. Actual rotation is adding a new secret version; because consumers reference the versionless secret, a rotated value flows through with no app/flow rebuild. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$VaultName, [string]$SecretName, # omit to apply to all secrets in the vault [string]$Owner, [string]$Purpose, [int]$RotationDays = 90, [datetime]$ExpiresOn, # overrides the computed today+RotationDays expiry [string]$SubscriptionId, [switch]$Execute # omit for -WhatIf (default safe mode))

PPKV-Validation.ps1

Purpose: Read-only validation for the "Standardize Power Platform Secrets in Azure Key Vault" solution. Checks three things on a named vault: 1) the vault uses the Azure RBAC permission model (not legacy access policies); 2) the Power Platform / Dataverse first-party service principal holds the 'Key Vault Secrets User' role on the vault (or a named secret); 3) a specified secret exists and is enabled (metadata)

so the reference will resolve. Makes no changes and prints NO secret values. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PPKV-Validation.ps1 -VaultName 'pp-secrets-prod-kv' Optional: -SecretName 'crm-api-key' -PowerPlatformAppId '' Requires Az.Accounts, Az.KeyVault, Az.Resources and Microsoft.Graph.Authentication / Microsoft.Graph.Applications (to resolve the Power Platform service principal). Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A PASS/FAIL line for each of the three checks. All PASS means Pattern A (Secret environment variables) can resolve secrets from this vault. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. The Power Platform / Dataverse first-party app can be identified by its well-known application ID; if your tenant differs, pass -PowerPlatformAppId explicitly. Operates on metadata only. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$VaultName, [string]$SecretName, [string]$SubscriptionId, # Well-known first-party app IDs seen for Power Platform / Dataverse Key Vault reference resolution. # Override if your tenant surfaces a different service principal. [string]$PowerPlatformAppId)

How to run: powershell -ExecutionPolicy Bypass -File .\PPKV-Validation.ps1 -VaultName 'pp-secrets-prod-kv' Optional: -SecretName 'crm-api-key' -PowerPlatformAppId '<app id>' Requires Az.Accounts, Az.KeyVault, Az.Resources and Microsoft.Graph.Authentication / Microsoft.Graph.Applications (to resolve the Power Platform service principal). Run Connect-AzAccount first, or the script will prompt. EXPECTED RESULTS A PASS/FAIL line for each of the three checks. All PASS means Pattern A (Secret environment variables) can resolve secrets from this vault. .NOTES Workplace Marketplace — Azure Solutions. Read-only. The repeatable way. The Power Platform / Dataverse first-party app can be identified by its well-known application ID; if your tenant differs, pass -PowerPlatformAppId explicitly. Operates on metadata only. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$VaultName, [string]$SecretName, [string]$SubscriptionId, # Well-known first-party app IDs seen for Power Platform / Dataverse Key Vault reference resolution. # Override if your tenant surfaces a different service principal. [string]$PowerPlatformAppId)